

Tutorium 4

Verzweigungen, Lokale Definitionen & Tupel

Grundlagen der Programmierung

15. Juni 2026

Verzweigungen in Haskell

Haskell bietet verschiedene Wege, um logische Pfade zu steuern.

if-then-else

```
1 signum If x =  
2   if x < 0  
3   then "neg"  
4   else "pos"
```

Guards (|)

```
1 signum G x  
2   | x < 0      = "  
3               neg"  
3   | otherwise = "  
4               pos"
```

case-of

```
1 signum C x =  
2   case compare x 0  
3     of  
4       LT -> "neg"  
        _  -> "pos"
```

Scope begrenzen: let & where

Globale Hilfsmethoden “verschmutzen” oft den Namensraum. Um Helper-Methoden privat zu halten, nutzen wir lokale Definitionen.

Mit `let ... in`

```
1 foo x =  
2   let helper y = y + 1  
3   in helper x
```

Die Definition steht **vor** der Nutzung.

Mit `where`

```
1 foo x = helper x  
2   where helper y = y + 1
```

Die Definition steht **nach** der Nutzung. (Oft in Haskell bevorzugt).

Tupel und Vektoren

Mit `type` kann man Datentypen (z. B. `Tupel`) benennen (Typ-Synonyme). Wir können direkt auf den `Tupel`-Inhalten Pattern Matching anwenden.

Beispiel: Kreuzprodukt

```
1  type Vector3D = (Double, Double, Double)
2
3  crossProduct :: Vector3D -> Vector3D -> Vector3D
4  crossProduct (x1, y1, z1) (x2, y2, z2) =
5      let cx = y1 * z2 - z1 * y2
6          cy = z1 * x2 - x1 * z2
7          cz = x1 * y2 - y1 * x2
8      in (cx, cy, cz)
```

Live Demo



`verzweigungen.hs, lokaleDefinitionen.hs, vector.hs`

Wir programmieren Guards, `where`-Blöcke und Vektor-Operationen zusammen im Code!